

Análise de Código Java

QuickSort — Ordenação por Particionamento

1. Objetivo Geral

O código implementa o algoritmo **QuickSort**, cujo propósito é ordenar um array de inteiros em ordem crescente de forma eficiente. No método `main`, um array de exemplo é inicializado, ordenado e impresso no console via `Arrays.toString()`.

2. Lógica e Algoritmo

A estratégia adotada é **dividir para conquistar**: o método `partition()` escolhe o *último elemento* como pivô e reorganiza o array de modo que todos os valores menores ou iguais ao pivô fiquem à sua esquerda e os maiores à sua direita, retornando o índice final do pivô. Em seguida, `quickSort()` chama a si mesmo recursivamente sobre os dois sub-arrays resultantes, até que cada segmento tenha tamanho zero ou um (`lo >= hi`), encerrando a recursão.

3. Conceitos Java Envolvidos

Conceito	Detalhe
Recursão	<code>quickSort()</code> chama a si mesmo sobre sub-arrays
Arrays primitivos	<code>int[]</code> passado por referência; alterações são in-place
Método estático	Sem instância de classe necessária
<code>java.util.Arrays</code>	Usado apenas para imprimir o resultado final

4. Complexidade

No **caso médio**, o QuickSort apresenta complexidade de tempo **$O(n \log n)$** , tornando-o muito eficiente na prática. No **pior caso** — array já ordenado com pivô sempre no extremo — a complexidade degrada para **$O(n^2)$** . O espaço adicional utilizado é **$O(\log n)$** para a pilha de chamadas recursivas; o algoritmo opera *in-place*, sem arrays auxiliares.

5. Decisões de Design e Limitações

A escolha do **último elemento como pivô fixo** é simples de implementar, porém torna o algoritmo vulnerável ao pior caso em arrays já ordenados ou quase ordenados. Alternativas como *pivô aleatório* ou *mediana de três* mitigariam esse risco. Além disso, a ausência de verificação de `null` ou array vazio pode gerar erros em chamadas descuidadas. Para uso em produção, recomenda-se `Arrays.sort()` da JDK, que emprega um DualPivot QuickSort

otimizado.

Trecho Principal

```
// Ponto de entrada – chama quickSort sobre o array inteiro
```

```
quickSort(arr, 0, arr.length - 1);
```

```
// Caso base da recursão
```

```
if (lo >= hi) return;
```

```
// Particionamento e chamadas recursivas
```

```
int p = partition(arr, lo, hi);
```

```
quickSort(arr, lo, p - 1); // sub-array esquerdo
```

```
quickSort(arr, p + 1, hi); // sub-array direito
```

Análise gerada por Claude · Anthropic